Word2Vec Demo: Reading the Output

A plain-language guide to what each line of word2vec_demo.py means | Naji El Kotob + AI

1. The big idea in one sentence

Word2Vec turns each word into a list of numbers (a **vector**, also called an **embedding**). Words that appear in the **same surroundings** end up with **similar numbers**.

The model never learns what a word means. It only notices which words keep each other company: "you know a word by the company it keeps."

2. The full output

This is what the script printed when it ran:

```
Vocabulary size: 20
Training pairs: 158
Epoch   0 | loss = 2.9920
Epoch  50 | loss = 1.8388
Epoch 100 | loss = 1.8317
Epoch 150 | loss = 1.8327
Epoch 200 | loss = 1.8319
Epoch 250 | loss = 1.8274
Epoch 299 | loss = 1.8323

--- Most similar words ---
king   -> queen (1.00), prince (0.87), princess (0.86)
cat    -> dog (0.99), mouse (0.87), drinks (0.38)
man    -> woman (0.82), prince (0.62), princess (0.60)
milk   -> the (0.43), drinks (0.27), chases (0.20)

--- Vector for 'king' (first 5 of 10 numbers) ---
[-0.639  1.096 -0.319  0.197 -0.511]
```

3. Setup lines

Line	What it means
`Vocabulary size: 20`	The 11 sample sentences contain 20 unique words, so the model learns 20 vectors (one per word).
`Training pairs: 158`	The number of practice examples. Each is a pair (center word, neighbor word). From "the king rules" we get (king, the) and (king, rules).

4. Loss: how wrong the guesses are

During training the model tries to guess a word's neighbors. **Loss** is a score for how wrong those guesses are. Lower is better. One **epoch** is one full pass over all 158 pairs.

Epoch	Loss	What is happening
`0`	2.99	Pure guessing. With 20 words, a random guess scores ln(20) = 2.996, almost exactly this. The model knows nothing yet.
`50`	1.84	Big drop. The model has already picked up most of the patterns.
`100–299`	~1.83	Flat. The model has learned all it can from this small text.

Why doesn't the loss reach 0? Because language is uncertain. After the word "the", the next word could be king, queen, cat or dog. No model can be 100% sure, so some error always remains. A flat loss means "done learning", not "broken".

5. Most similar words

The model compares word vectors using **cosine similarity**, which checks whether two vectors point in the same direction.

Score	Meaning
`1.00`	Same direction: the words are used in the same way
`around 0.5`	Somewhat related
`0`	Unrelated
`-1.00`	Opposite direction

Result	Why
`king → queen (1.00)`	They sit in identical sentences: "the ___ rules the kingdom" and "the ___ is a …". Same company, so the same vector. Prince and princess follow because they share the "the ___ is a" pattern.
`cat → dog (0.99)`	Same story: "the ___ drinks milk" and "the ___ chases …". Mouse is close too because it also appears near "chases".
`man → woman (0.82)`	Similar, but less than king/queen. Their sentences differ a bit ("is a man" vs "is a woman", "a man and a woman walk").
`milk → the (0.43)`	Weak. "milk" appears only twice, always as the last word, so the model has very little to learn from.

Lesson from "milk": Word2Vec needs a lot of text. Rare words get poor vectors. Real models train on billions of words, not 11 sentences.

6. The vector for "king"

```
[-0.639  1.096 -0.319  0.197 -0.511]
```

- These are the first 5 of the 10 numbers that now **represent** the word king.
- A single number has no meaning on its own. You can't say "number 2 means royalty".
- The numbers only become useful when compared with other words' vectors, which is exactly what the similarity scores in section 5 do.

7. Key takeaways

- **Similar context = similar vector.** King and queen are close because they appear in the same places, not because the model "understands" royalty.
- **Loss falls fast, then flattens.** Learning stops once the model has captured the patterns in the data.
- **More text = better vectors.** Rare words like "milk" are the weak spot of a small corpus.
- **This is the ancestor of modern AI embeddings.** Today's embedding models (used in RAG and vector databases) follow the same idea, but read whole sentences and give different vectors for different meanings of a word.